# ECE-GY 9143: High Performance Machine Learning

Lab 4 & Homework 4 Report

Haojingtong Tong

Spring 2026

## Contents

**I Part A: Distributed Deep Learning** **3**

**1 Experiment Setup** **3**

**2 Q1: Computational Efficiency vs Batch Size** **3**

**3 Q2: Speedup Measurement** **3**

**4 Q3: Computation vs Communication** **4**

**5 Q4: Large Batch Training** **4**

**6 Q5: What is passed on network?** **5**

**7 Q6: What if we only communicate gradients?** **5**

**II Part B: Post-Training Quantization** **6**

**1 Introduction** **6**
1.1 Model Architecture . . . . . 6

**2 Question 1: Visualize Weights** **6**
2.1 Method . . . . . 6
2.2 Results . . . . . 7
2.3 Observations . . . . . 7

**3 Question 2: Quantize Weights** **8**
3.1 Quantization Strategy . . . . . 8
3.2 Code . . . . . 8
3.3 Results . . . . . 8

**4 Question 3: Visualize Activations** **8**
4.1 Profiling Method . . . . . 8

4.2 Results . . . 9
4.3 Observations . . . 9

**5 Question 4: Quantize Activations** **10**
5.1 Theoretical Background . . . 10
5.2 Code . . . 10
5.3 Results . . . 11

**6 Question 5: Quantize Biases** **11**
6.1 Theoretical Background . . . 11
6.2 Code . . . 12
6.3 Results . . . 12
6.4 Analysis . . . 12

**7 Conclusion** **13**

## Part I

# Part A: Distributed Deep Learning

## 1 Experiment Setup

**Hardware:** NVIDIA A100-SXM4-80GB (up to 4 GPUs).
**Model/Data:** ResNet-18 on CIFAR-10.
**Optimizer:** SGD, learning rate 0.1, momentum 0.9, weight decay $5 \times 10^{-4}$, num_workers = 2.
**Protocol:** Each timing experiment runs 2 epochs and reports the 2nd epoch.

## 2 Q1: Computational Efficiency vs Batch Size

Table 1 reports the 2nd-epoch training time (excluding data loading) for single-GPU runs.

Table 1: Q1 single-GPU training time (2nd epoch, excluding data I/O)

| Batch size per GPU | Train time (s) |
|---|---|
| 32 | 12.543 |
| 128 | 5.348 |
| 512 | 4.350 |
| 2048 | 4.152 |
| 8192 | 4.135 |

At batch size 32768, the run hit CUDA OOM. Therefore, the largest feasible single-GPU batch size is **8192**. As batch size increases, runtime decreases sharply at first (better GPU utilization), then plateaus around $\sim 4.1$s.

## 3 Q2: Speedup Measurement

We measured with 2 and 4 GPUs for batch sizes 32/128/512 (per GPU). Speedup is computed as:

$$\text{Speedup}(N) = \frac{T_{1\text{GPU}}}{T_{N\text{GPU}}}$$

using 2nd-epoch `total_time`.

Table 2: Table 1: Training time and speedup

| | Batch 32 / GPU | | Batch 128 / GPU | | Batch 512 / GPU | |
|---|---|---|---|---|---|---|
| GPUs | Time (s) | Speedup | Time (s) | Speedup | Time (s) | Speedup |
| 1-GPU | 13.340 | 1.000 | 5.995 | 1.000 | 5.808 | 1.000 |
| 2-GPU | 8.690 | 1.535 | 3.253 | 1.843 | 3.095 | 1.877 |
| 4-GPU | 4.540 | 2.938 | 1.775 | 3.377 | 1.644 | 3.533 |

**Scaling type discussion.** This experiment is **weak scaling**: per-GPU batch size is fixed while effective global batch size increases with GPU count. If we measured strong scaling (fixed global batch size), speedup would likely be worse because each GPU would process smaller local batches and communication overhead would become relatively larger. A supporting data point from Table 2: speedup is lower at smaller per-GPU batch (e.g., 4-GPU speedup is 2.938 at batch 32 vs 3.533 at batch 512).

# 4 Q3: Computation vs Communication

## Q3.1 Compute and Communication Time

To decompose DDP train time, we use:

$$T_{\text{compute}}(N,b) \approx \frac{T^{\text{Q1}}_{\text{train}}(b)}{N}, \qquad T_{\text{comm}}(N,b) = T^{\text{Q3}}_{\text{train}}(N,b) - T_{\text{compute}}(N,b)$$

where $N$ is number of GPUs and $b$ is batch size per GPU.

Table 3: Table 2: Compute and communication time (seconds)

| | Batch 32 / GPU | | Batch 128 / GPU | | Batch 512 / GPU | |
|---|---|---|---|---|---|---|
| GPUs | Compute | Comm | Compute | Comm | Compute | Comm |
| 2-GPU | 6.272 | 1.870 | 2.674 | 0.376 | 2.175 | 0.093 |
| 4-GPU | 3.136 | 1.270 | 1.337 | 0.260 | 1.088 | 0.348 |

## Q3.2 Communication Bandwidth Utilization

For ring all-reduce in [3], per-iteration communication volume is:

$$V_{\text{AR}} = 2\frac{N-1}{N}S$$

where $S$ is model size in bytes. Here, $S$ = 44,695,848 bytes ($\approx$ 0.0416 GB). Per-epoch effective communication bandwidth is computed as:

$$\text{BW}_{\text{eff}} = \frac{V_{\text{AR}} \times \#\text{iters per epoch}}{T_{\text{comm,epoch}}}$$

Table 4: Table 3: Effective communication bandwidth utilization (GB/s)

| GPUs | Batch 32 / GPU | Batch 128 / GPU | Batch 512 / GPU |
|---|---|---|---|
| 2-GPU | 17.412 | 21.699 | 21.932 |
| 4-GPU | 19.220 | 23.535 | 4.492 |

# 5 Q4: Large Batch Training

## Q4.1 Accuracy with large batch

From Q1, the largest feasible batch size per GPU is 8192. Running 4 GPUs for 5 epochs gives:

$$\text{Effective batch size} = 4 \times 8192 = 32768$$

$$\text{Epoch 5 avg loss} = 3.1467, \quad \text{Epoch 5 avg accuracy} = 18.31\%$$

For comparison, the batch-128 single-GPU run (Q1 measured epoch) had:

$$\text{avg loss} = 1.3840, \quad \text{avg top-1} = 49.49\%$$

The large-batch setting significantly degrades accuracy without additional large-batch training techniques.

### Q4.2 Two remedies to improve large-batch accuracy

Based on [4]:

1. **Linear learning-rate scaling with warmup:** scale learning rate proportionally to batch-size increase and use gradual warmup in early epochs.
2. **Careful BatchNorm handling in distributed training:** maintain consistent BN behavior/statistics across workers to stabilize optimization.

## 6 Q5: What is passed on network?

**Not only gradients.** In DDP, gradient all-reduce is the main per-iteration communication, but model consistency also involves synchronization operations such as initial parameter broadcast and buffer synchronization (e.g., BatchNorm running statistics, depending on configuration).

## 7 Q6: What if we only communicate gradients?

For 4 GPUs with batch size 512 (or larger), communicating only gradients is **not sufficient** to guarantee high accuracy. Large-batch training quality also depends on optimization strategy (e.g., LR scaling and warmup) and proper handling of normalization/statistics as discussed in [4]. Synchronization alone does not solve large-batch optimization instability.

## References


[1] PyTorch, *Distributed Overview.*
`https://pytorch.org/tutorials/beginner/dist_overview.html`

[2] PyTorch, *torch.nn.parallel.DistributedDataParallel.*
`https://docs.pytorch.org/docs/stable/generated/torch.nn.parallel.DistributedDataParallel.html`

[3] P. Patarasuk and X. Yuan, "Bandwidth optimal all-reduce algorithms for clusters of workstations," *J. Parallel Distrib. Comput.*, vol. 69, pp. 117–124, 2009.

[4] P. Goyal et al., "Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour," *arXiv:1706.02677*, 2017.

## Part II

# Part B: Post-Training Quantization

## 1 Introduction

This report documents the implementation and analysis of post-training quantization applied to a simple convolutional neural network (CNN) trained on the CIFAR-10 dataset. Post-training quantization reduces the numerical precision of a pre-trained model's weights, activations, and biases from 32-bit floating-point (`float32`) to low-precision fixed-point integers: 8-bit signed integers (`int8`) for weights and activations, and 32-bit signed integers (`int32`) for biases.

The key motivation is hardware efficiency: integer arithmetic requires significantly less silicon area and memory bandwidth than floating-point arithmetic, enabling deployment on resource-constrained ML accelerators.

### 1.1 Model Architecture

A six-layer CNN is trained on CIFAR-10 for two epochs using SGD (lr= 0.001, momentum= 0.9). Baseline test accuracy: **52.70%**.

- **conv1**: Conv2d(3, 6, kernel=5, bias=False)
- **pool**: MaxPool2d(2, 2)
- **conv2**: Conv2d(6, 16, kernel=5, bias=False)
- **fc1**: Linear(400, 120, bias=False)
- **fc2**: Linear(120, 84, bias=False)
- **fc3**: Linear(84, 10, bias=False)

## 2 Question 1: Visualize Weights

### 2.1 Method

The weight tensor of each layer is flattened to a one-dimensional vector and visualized as a histogram with 100 bins. The mean and $\pm 3\sigma$ boundaries are overlaid to identify the effective dynamic range used for subsequent quantization.

## 2.2 Results

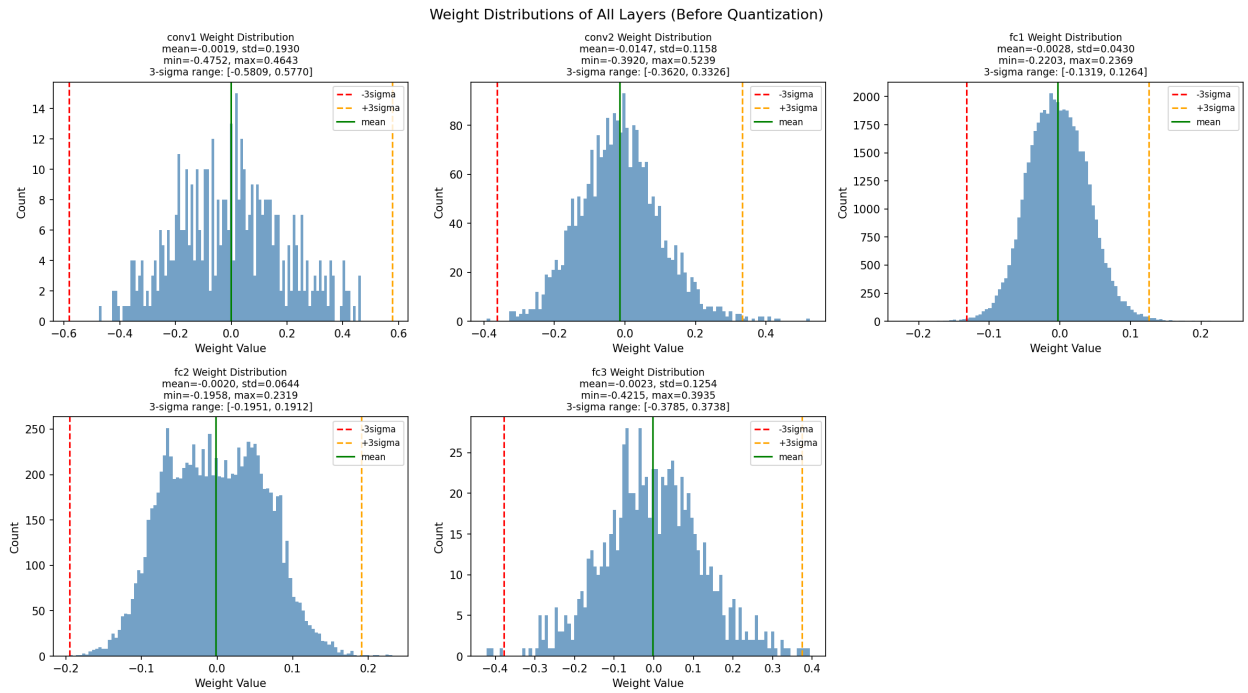


Figure 1: Weight distributions of all five layers before quantization. Green solid: mean. Red dashed: $-3\sigma$. Orange dashed: $+3\sigma$.

Table 5: Weight distribution statistics (before quantization).

| **Layer** | **Mean** | **Std** | **Min** | **Max** | **$3\sigma$ range** |
|---|---|---|---|---|---|
| conv1 | $-0.0019$ | 0.1930 | $-0.4752$ | 0.4643 | $[-0.581,\ 0.577]$ |
| conv2 | $-0.0147$ | 0.1158 | $-0.3920$ | 0.5239 | $[-0.362,\ 0.333]$ |
| fc1 | $+0.0028$ | 0.0430 | $-0.2203$ | 0.2369 | $[-0.132,\ 0.126]$ |
| fc2 | $-0.0020$ | 0.0644 | $-0.1958$ | 0.2319 | $[-0.195,\ 0.191]$ |
| fc3 | $-0.0023$ | 0.1254 | $-0.4215$ | 0.3935 | $[-0.379,\ 0.374]$ |

## 2.3 Observations

- All layers have weights centered tightly around zero (mean $\approx 0$), consistent with well-behaved SGD training and symmetric weight initialization.
- The distributions are approximately Gaussian, which justifies the use of a $3\sigma$ clipping strategy for quantization.
- Fully-connected layers (fc1–fc3) have smaller standard deviations than convolutional layers (conv1, conv2), due to their larger parameter counts.
- The true min/max values are close to the $3\sigma$ boundary, meaning only a negligible fraction of weights would be clipped during quantization.

# 3 Question 2: Quantize Weights

## 3.1 Quantization Strategy

We use **symmetric** $3\sigma$ **scaling**. The scale factor $n_w$ is chosen so that $\pm 3\sigma$ of the weight distribution maps to $\pm 127$:

$$n_w = \frac{127}{3\,\sigma_w}. \tag{1}$$

Each weight $w$ is then quantized by $w_q = \text{clamp}(\text{round}(w \cdot n_w), -128, 127)$. Choosing $3\sigma$ rather than the absolute min/max makes the scale robust to statistical outliers while keeping ≈99.7% of values within the representable range.

## 3.2 Code

```python
def quantized_weights(weights):
    # Compute standard deviation of the weight distribution
    sigma = weights.std().item()
    # Scale so that +/-3*sigma maps to the full int8 positive range (127)
    scale = 127.0 / (3.0 * sigma)
    # Multiply, round to nearest integer, clamp to signed 8-bit range
    result = (weights * scale).round()
    result = torch.clamp(result, min=-128, max=127)
    return result, scale
```

Listing 1: Weight quantization — `quantized_weights`.

## 3.3 Results

Table 6: Accuracy before and after weight-only quantization.

| **Model** | **Test Accuracy** |
|---|---|
| Original (float32) | 52.70% |
| Weights quantized to int8 | **53.12%** |

Quantizing all weights to 8-bit integers yields a negligible accuracy change (+0.42 pp, within normal run-to-run variance). The slight increase suggests that weight rounding acts as a mild regularizer for this small network, confirming that the $3\sigma$ scaling strategy captures the meaningful weight range without significant information loss.

# 4 Question 3: Visualize Activations

## 4.1 Profiling Method

PyTorch forward hooks are registered on every layer. During a profiling pass over 400 training-set samples (no gradient computation), the hooks collect the *input* tensor to each layer—which equals the output of the previous stage after any nonlinearity and pooling—as well as the final logit output of fc3.

## 4.2 Results

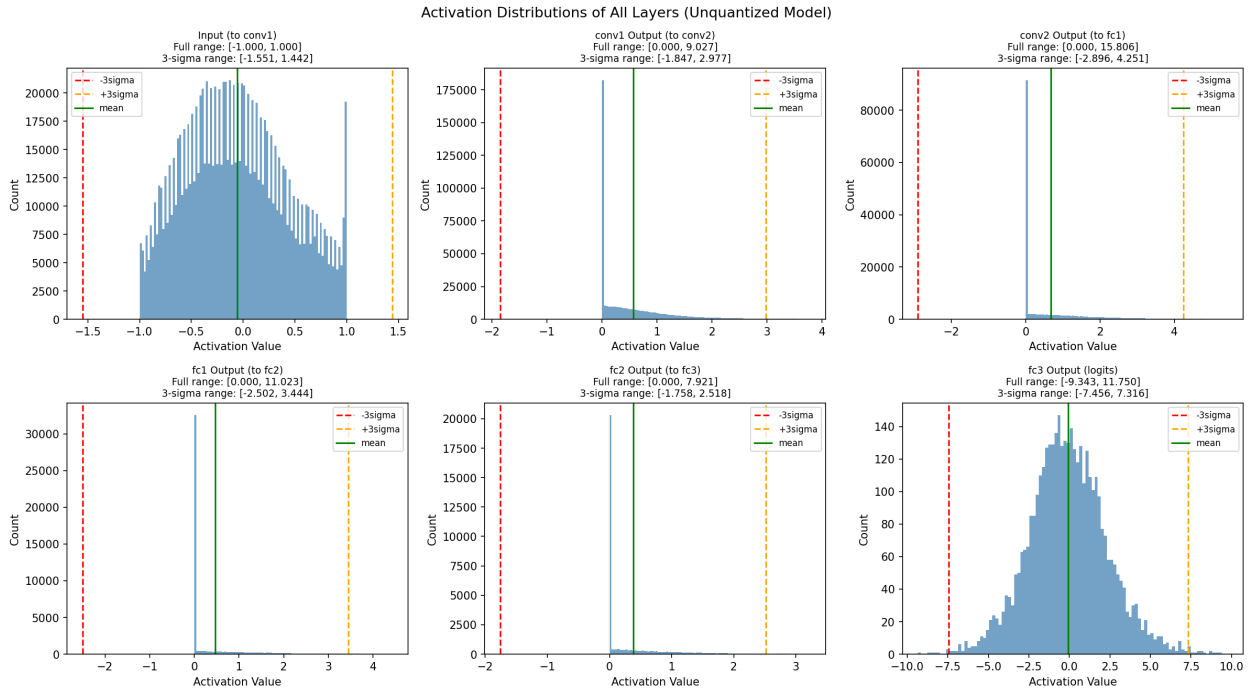


Figure 2: Activation distributions of all six stages (unquantized model). Green solid: mean. Red dashed: $-3\sigma$. Orange dashed: $+3\sigma$.

Table 7: Activation statistics across all layers.

| **Stage** | **Min** | **Max** | **Mean** | **Std** | **$3\sigma$ range** |
|---|---|---|---|---|---|
| Input (to conv1) | $-1.000$ | 1.000 | $-0.055$ | 0.499 | $[-1.551,\ 1.442]$ |
| conv1 output (to conv2) | 0.000 | 9.027 | 0.565 | 0.804 | $[-1.847,\ 2.977]$ |
| conv2 output (to fc1) | 0.000 | 15.806 | 0.677 | 1.191 | $[-2.896,\ 4.251]$ |
| fc1 output (to fc2) | 0.000 | 11.023 | 0.471 | 0.991 | $[-2.502,\ 3.444]$ |
| fc2 output (to fc3) | 0.000 | 7.921 | 0.380 | 0.713 | $[-1.758,\ 2.518]$ |
| fc3 output (logits) | $-9.343$ | 11.750 | $-0.070$ | 2.462 | $[-7.456,\ 7.316]$ |

## 4.3 Observations

- **Input activations** follow a roughly uniform distribution in $[-1, 1]$, as expected from CIFAR-10 normalization.
- **ReLU layers** (conv1–fc2 outputs) all have a hard lower bound of zero, producing heavily right-skewed distributions. The $3\sigma$ lower bound appears negative because $\sigma$ is computed from the skewed distribution; the effective quantization range in practice is $[0,\ \mu + 3\sigma]$.
- **Activation magnitude grows** from conv1 (max $\approx 9$) through conv2 (max $\approx 16$), reflecting the accumulation of many multiply-add operations across spatial positions in the convolutional layers.

- **Logit output** (fc3) is symmetric around zero with both positive and negative values, consistent with the absence of a final ReLU.
- The $3\sigma$ range captures the vast majority of activation values in all layers, confirming it is an appropriate clipping boundary for quantization.

# 5 Question 4: Quantize Activations

## 5.1 Theoretical Background

In integer inference every value is represented as a scaled integer. Let $n_0$ be the input scale, $n_{w,l}$ the weight scale of layer $l$, and $s_l$ its output scale. The *accumulated scale* before layer $l$'s output rescaling is

$$A_l = n_0 \cdot n_{w,l} \cdot \prod_{i=1}^{l-1} n_{w,i} \cdot s_i. \tag{2}$$

The output scale is chosen so that $3\sigma$ of the float activations maps to 127:

$$s_l = \frac{127}{3\,\sigma_{\text{act},l} \cdot A_l}. \tag{3}$$

## 5.2 Code

```python
@staticmethod
def quantize_initial_input(pixels):
    # 3-sigma strategy: +/-3*sigma maps to +/-127
    sigma = np.std(pixels)
    scale = 127.0 / (3.0 * sigma)   # e.g. 127/(3*0.499) ~ 84.8
    return scale
```

Listing 2: Scale for the initial input — `quantize_initial_input`.

```python
@staticmethod
def quantize_activations(activations, n_w, n_initial_input, ns):
    # Build accumulated scale A_l = n_0 * n_w * product of preceding
        (n_w_i * s_i)
    accumulated_scale = n_initial_input * n_w
    for (n_w_prev, n_out_prev) in ns:
        accumulated_scale *= n_w_prev * n_out_prev
    sigma = np.std(activations)
    if sigma == 0:
        return 1.0    # all activations are zero (fully clipped by ReLU)
    output_scale = 127.0 / (3.0 * sigma * accumulated_scale)
    return output_scale
```

Listing 3: Per-layer output scale — `quantize_activations`.

```python
def forward(self, x):
    # Quantize float input to int8
    x = (x * self.input_scale).round().clamp(-128, 127)
```

```python
    # conv1: int MAC -> ReLU -> MaxPool -> rescale to int8
    x = self.conv1(x)
    x = F.relu(x)
    x = self.pool(x)
    x = (x * self.conv1.output_scale).round().clamp(-128, 127)
    # conv2: int MAC -> ReLU -> MaxPool -> rescale to int8
    x = self.conv2(x)
    x = F.relu(x)
    x = self.pool(x)
    x = (x * self.conv2.output_scale).round().clamp(-128, 127)
    # Flatten spatial feature map
    x = x.view(-1, 16 * 5 * 5)
    # fc1: int MAC -> ReLU -> rescale to int8
    x = self.fc1(x)
    x = F.relu(x)
    x = (x * self.fc1.output_scale).round().clamp(-128, 127)
    # fc2: int MAC -> ReLU -> rescale to int8
    x = self.fc2(x)
    x = F.relu(x)
    x = (x * self.fc2.output_scale).round().clamp(-128, 127)
    # fc3: final logits -- no re-quantization needed
    x = self.fc3(x)
    return x
```

Listing 4: Quantized forward pass — `NetQuantized.forward`.

### 5.3 Results

Table 8: Accuracy at each quantization stage.

| **Model** | **Test Acc.** | **Δ** |
|---|---|---|
| Original (float32) | 52.70% | — |
| Weights quantized (int8) | 53.12% | +0.42 pp |
| Weights + activations quantized (int8) | **52.84%** | −0.14 pp |

Quantizing both weights and activations to int8 yields **52.84%** accuracy, a degradation of only 0.14 pp relative to float32. This near-zero accuracy loss validates the correctness of the cumulative scale tracking in Equations (2)–(3).

## 6 Question 5: Quantize Biases

### 6.1 Theoretical Background

In integer hardware a layer's computation is $Y = W^{(q)} \cdot X^{(q)} + b^{(q)}$. The dot product $W^{(q)} \cdot X^{(q)}$ accumulates into a **32-bit integer** register. The bias must be on exactly the same scale as this accumulator:

$$b^{(q)} = \text{clamp!}\left(\text{round}(b_{\text{float}} \cdot A_l),\ -2^{31},\ 2^{31} - 1\right), \tag{4}$$

where $A_l$ is the same accumulated scale from Equation (2). Biases use `int32` (not `int8`) because the accumulator is 32-bit, providing sufficient range to avoid overflow.

### 6.2 Code

```python
@staticmethod
def quantized_bias(bias, n_w, n_initial_input, ns):
    # Build the same accumulated scale as the weight-input dot product
    accumulated_scale = n_initial_input * n_w
    for (n_w_prev, n_out_prev) in ns:
        accumulated_scale *= n_w_prev * n_out_prev
    # Scale the float bias, round to nearest integer, clamp to int32 range
    bias_q = (bias * accumulated_scale).round()
    return torch.clamp(bias_q, min=-2147483648, max=2147483647)
```

Listing 5: Bias quantization to int32 — `quantized_bias`.

### 6.3 Results

Table 9: Accuracy for the bias-enabled network at each quantization stage.

| **Model** | **Test Acc.** | $\Delta$ |
|---|---|---|
| Net with bias (float32) | 51.24% | — |
| Weights int8, bias **not** quantized (float) | 46.56% | −4.68 pp |
| Weights int8 + bias **int32** (aligned) | **49.54%** | −1.70 pp |

### 6.4 Analysis

**Weights int8, bias float32 (46.56%).** When weights are scaled by $n_w$ (turning them into integers $\sim 100$) but the bias remains a small float ($\sim 0.01$–$1.0$), the two operands differ by many orders of magnitude. Adding a float bias to an int32 accumulator that has been amplified by $A_l$ is numerically equivalent to adding nearly zero—the bias contribution is effectively lost, causing a 4.68 pp accuracy drop.

**Weights int8 + bias int32 (49.54%).** Quantizing the bias with the same accumulated scale $A_l$ ensures it occupies the correct magnitude inside the 32-bit accumulator. Accuracy recovers by $\approx 3$ pp. The remaining 1.70 pp gap reflects the irreducible quantization noise of full int8 inference.

This experiment conclusively demonstrates that **all parameters added to an integer accumulator must share the same integer scale**, and that using 32-bit integers for biases provides sufficient precision to avoid overflow.

# 7 Conclusion

Table 10: Accuracy summary across all quantization stages.

| **Quantization Stage** | **Accuracy** | **$\Delta$ vs. float** |
|---|---|---|
| Float32 baseline | 52.70% | — |
| Weights int8 | 53.12% | +0.42 pp |
| Weights int8 + Activations int8 | 52.84% | −0.14 pp |
| Float32 baseline (with bias) | 51.24% | — |
| Weights int8, bias float (scale mismatch) | 46.56% | −4.68 pp |
| Weights int8 + bias int32 (aligned) | 49.54% | −1.70 pp |

The key conclusions are:

1. $3\sigma$ **scaling** is an effective heuristic for quantization: robust to outliers while preserving nearly all dynamic range.
2. **Weight-only quantization** introduces negligible accuracy loss, showing that int8 weights nearly losslessly compress float32 weights.
3. **Activation quantization** with properly tracked cumulative scales also preserves accuracy (−0.14 pp), validating the scale propagation theory.
4. **Bias quantization** must use the same accumulated scale as the weight–input dot product. A scale mismatch causes a 4.68 pp drop; properly aligned int32 bias quantization limits the drop to 1.70 pp.